



Project OS — Guide complet d'utilisation

v0.1.0 Pilotage de projets par commandes, moteur IA local (LocalAI). Ce guide se passe de l'IDE Cline : on utilise **les commandes slash et les fichiers** directement.

Prérequis GPU (obligatoire pour la ladder). Project OS exige un vrai GPU. Pas de fallback CPU pour la validation d'endurance. Vérifier : `nvidia-smi --query-gpu=memory.free` → il faut > ≈3,0 Go libres pour un modèle Q4 2–3 Go. Seul l'utilisateur peut fermer les applications GPU externes (jamais de kill automatique de process).

1. Où est installé Project OS ? (chemins complets)

Élément	Chemin complet
Racine projet (code)	<code>C:\Users\eiden\Desktop\dev\projet-os</code>
Projets gérés	<code>C:\Users\eiden\Desktop\dev\projects\<slug></code>
Config VS Code / Antigravity	<code>C:\Users\eiden\.vscode\argv.json</code>
Modèles LocalAI	<code>C:\Users\eiden\LocalAI\models\llama-cpp\...</code>
Artefacts d'endurance	<code>C:\Users\eiden\Desktop\dev\projet-os\artifacts\endurance</code>
Tests	<code>...\projet-os\src\tests</code>
Cible LocalAI	<code>http://127.0.0.1:8080/v1</code>

2. Commandes slash (contrôle des projets)

Elles s'utilisent dans la console d'input de Project OS (extension VS Code/Antigravity), préfixées par `/`. Types de projets pilotés : `cpp`, `ts`, `web`, `node`.

Commande	Description	Exemple
<code>/create</code>	Crée un projet neuf (scaffold).	<code>/create myproject type=cpp</code>
<code>/goal</code>	Définit l'objectif (critères mesurables).	<code>/goal "Observe Swiss Super League" showStandings showScorers</code>
<code>/todo</code>	Liste (barré) / ajoute / marque une tâche.	<code>/todo list</code> , <code>/todo done impl</code>
<code>/addon</code>	Gère les addons (core, cpp, web, node).	<code>/addon add project-os-cpp</code>
<code>/autonomy</code>	Lance un run autonome (analyse/enrichissement).	<code>/autonomy plan --minutes=60</code> , <code>/autonomy run</code>
<code>/docs</code>	Navigation des sources officielles en ligne.	<code>/docs swiss football league</code>
<code>/project</code>	Résumé d'un projet géré.	<code>/project sfl-observatory</code>

`/report`

Rapport tokens/cost/perf d'un job.

`/report --job=run-60min`

3. Créer & piloter un projet (exemple réel SFL)

```
cd C:\Users\eiden\Desktop\dev\projects
/create sfl-observatory type=cpp
/goal "Observe Swiss Super League" showStandings showScorers
/todo add "Implement standings/scorers"
/todo done scaffold
/autonomy plan --minutes=60
/autonomy run          # Project OS analyse et enrichit (write-lane)
/docs swiss football league # navigation sources en ligne
/project sfl-observatory # état du projet
/report --job=run-60min  # tokens + coût + perf
```

4. Anatomie d'un projet géré

```
<slug>
├─ project.json          # métadonnées (id, type, status)
├─ goal.json             # objectif + critères + progress
├─ todo.json             # tâches durable (persisté par /create)
├─ .project-os\
│   ├─ autonomy.json     # plan/mode autonomy
│   ├─ handoff.md        # note de continuation
│   └─ addons.lock.json  # addons activés
├─ .agents\
│   ├─ rules<type>.md    # règles (cpp.md...)
│   └─ skills<type>-dev\SKILL.md
├─ src\                  # code
├─ tests\                # tests (ctest / node)
└─ docs\
    └─ docs-nav.md       # navigation doc en ligne (via /docs)
```

5. Vérifier la santé (baseline) sans GPU

```
cd C:\Users\eiden\Desktop\dev\projet-os
npm run compile          # build TS
npm run typecheck        # 0 erreur
npm test                 # tous les tests PASS
```

6. GUI + build C++ (Win32/CMake)

```
$ENV:Path='C:\msys64\mingw64\bin;' + $ENV:Path
cd C:\Users\eiden\Desktop\dev\projects\<slug>
cmake -S . -B build -G "MinGW Makefiles"
```

```
cmake --build build
ctest --test-dir build --output-on-failure
```

7. GPU & ladder d'endurance (ne démarre qu'avec GPU prouvé)

Le timer de test ne démarre QUE si `GPU_OFFLOAD_PROOF=PASS` (delta VRAM + monitor + smoke). Un modèle `CPU_ONLY` ou `MODEL_READY_CPU` n'est pas autorisé.

Palier	Durée	Condition
GATE_1→GATE_5	5 / 10 / 20 / 30 / 60 min	GPU offload prouvé, 0 bug, 0 fallback CPU

```
node C:\Users\eiden\Desktop\dev\projet-os\scratch\endurance-run.mjs
```

8. Règles de sécurité non négociables

- **Jamais** de kill de process ni de reset Docker/Windows ; l'utilisateur contrôle les apps GPU externes.
- **Jamais** de CPU fallback, de PASS simulé, ni de qualification inventée.
- Licence ≤ Apache/4B, SHA vérifiée, Defender "NO KNOWN MALWARE INDICATORS".
- Flag `trust_remote_code=false` obligatoire (BLOCKED si requis).

9. Dépannage rapide

Symptôme	Cause	Action
<code>BLOCKED_GPU_EXTERNAL_VRAM</code>	<3 Go free	Fermer navigateur/IDE GPU-accéléré, débrancher écran 2 sur le dGPU
OOM à ctx long	Contexte géant (131072)	Réduire <code>context_size</code> à 8192 (2,8 Go vs 7,9 Go)
sfl.ch 404 / football-data 403	Chemin/clé requise	Corriger le chemin ou fournir la clé API

10. Télémétrie des jobs

Chaque run écrit son rapport : `artifacts\endurance\*.json` / `artifacts\sfl\AUTONOMY_USAGE_REPORT.json` — tokens (input/output/total), coût (LocalAI = `EXACT_ZERO`), perf (TTFT, tokens/s, VRAM).